Open-Domain Question Answering: From Lexical Retrieval to RAG

NLP (CS60075) Assignment II, Project 10 – Natural Questions (Open)

Saksham Kumar (24CS10029)

October 31, 2026

Abstract

This report describes three open-domain question answering systems evaluated on a fixed sample of 300 questions from Natural Questions (Open), over a Wikipedia-paragraph index of 45,029 passages built for this project. Task 1 is a classical pipeline using BM25 retrieval and rule-based answer extraction. Task 2 reproduces the retrieve-then-generate design of Retrieval-Augmented Generation (RAG) using a MiniLM dense retriever and the Qwen2.5-1.5B-Instruct generator. Task 3 fuses both rankings with Reciprocal Rank Fusion (RRF) and adds a lexical groundedness check. The dense RAG system performed best, with 27.7% Exact Match (EM) and 68.7% recall@10. The hybrid system did not improve on it, reaching 25.7% EM and 62.7% recall@10. The classical system retrieved the answer within the top 10 passages for 56.7% of questions, but achieved only 0.3% EM. The groundedness check flagged only wrong answers, but caught just 11.7% of them. We analyse why fusion did not help and why vocabulary overlap is a weak signal of correctness.

1 Introduction

Open-domain question answering (QA) requires a system to answer a question using a large document collection instead of a single passage supplied with the question. The problem has two parts: *retrieval*, which finds relevant evidence, and *answering*, which produces a short answer from that evidence. Chen et al.'s DrQA [**?**] established this retrieve-then-read design over Wikipedia, combining a TF-IDF document retriever with a neural span reader. It serves as the conceptual starting point for this project.

The three tasks vary each part in turn:

- **Task 1** uses lexical retrieval and extractive answering, with no neural components.
- **Task 2** uses dense retrieval and a generative language model.
- **Task 3** fuses the lexical and dense rankings, and uses lexical overlap as a groundedness check on generated answers.

Figure **??** shows how the components relate. All three systems share one passage index, one question sample, and one evaluation code path, so differences between them come from the methods and not from the data.

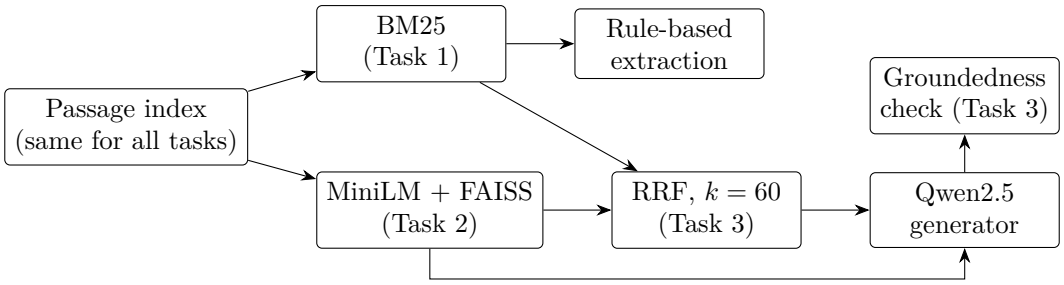


Figure 1: The three systems. Task 2 passes dense results directly to the generator, while Task 3 passes it the fused ranking.

2 Dataset and Passage Index

2.1 Question sample

We used `google-research-datasets/nq_open`, the open-domain variant of Natural Questions [**?**, **?**], in which each question has one or more acceptable short answers and no supplied passage. As the assignment specifies, we drew a fixed sample of $n = 300$ questions from the validation split, using `numpy.random.RandomState(42)` and sampling without replacement. The seed is 42 throughout. We used no training examples, so generation is zero-shot and no model was trained or fine-tuned.

2.2 Index construction

Natural Questions has no passage collection of its own, so we built one from English Wikipedia using the live MediaWiki API (`prepare_index.py`). The index is built once, then reused unchanged for all three tasks.

Page selection. Each question is searched twice, and the top three pages are kept from each search: once using the question as written, and once using its keywords joined with `OR`. Keywords are the question's lowercase word tokens after removing scikit-learn's English stopword list. The second query helps find pages worded differently from the question. A further 300 randomly chosen non-redirect pages act as distractors that the retrievers must filter out. **Gold answers are never used to select pages or write passages**; they are read only after the index is written, to compute a coverage statistic.

Segmentation. Plain-text page extracts are split into passages of about 100–200 words without crossing section boundaries:

1. Sections titled References, External links, See also, Further reading, Notes, Bibliography, Sources, Citations, Footnotes, or Works cited are skipped, including their subsections.
2. Lines are accumulated until a block reaches at least 100 words, or a section ends.
3. Blocks over 200 words are split at sentence boundaries, and any single sentence over 200 words is cut outright.
4. Chunks shorter than 30 words are dropped. Section-final chunks of 30–99 words are kept, so a minority of passages fall below the nominal 100-word minimum.

Reliability and reproducibility. Every API response is cached on disk so that an interrupted build can resume. Requests are spaced 1 s apart and retried up to eight times with exponential

backoff, following the server's `Retry-After` header when present, because the shared cluster IP address received HTTP 429 responses. Because Wikipedia changes and distractor pages are random, a rebuild would produce a different index. Reported results therefore apply to this frozen index file, whose build details are recorded in an accompanying manifest.

Index statistics. Table **??** summarises the index, built on 31 October 2026. Answer coverage is the share of sampled questions with a gold answer appearing anywhere in the index, and so is the upper limit on recall@k for any retriever. It is 86.3%, meaning that for 41 of the 300 questions no retriever could find a gold answer string.

Statistic	Value
Pages selected from question searches	1,552
Random distractor pages	300
Total pages	1,852
Total passages	45,029
Answer coverage	86.3% (259 of 300)

Table 1: Statistics of the frozen index, from its build manifest.

3 Evaluation Metrics

All answers and gold strings are normalised in the SQuAD style [**?**]: lowercasing, then removing punctuation and the articles *a*, *an*, *the*, then collapsing whitespace. With several acceptable answers, the best-scoring one counts.

Exact Match (EM) is 1 when the normalised prediction equals any normalised gold answer, and 0 otherwise.

Token F1 is the harmonic mean of token precision and recall, computed on token multisets:

$$P = \frac{|T_{\text{pred}} \cap T_{\text{gold}}|}{|T_{\text{pred}}|}, \quad R = \frac{|T_{\text{pred}} \cap T_{\text{gold}}|}{|T_{\text{gold}}|}, \quad F_1 = \frac{2PR}{P+R}. \tag{1}$$

Recall@k is the fraction of questions where a normalised gold answer appears as whole words in at least one of the top k passages, for $k \in \{1, 3, 5, 10\}$. Whole-word matching stops "no" from matching inside "know". Natural Questions provides no gold passages, so this is a string-presence measure: a passage can contain the answer string without actually supporting the answer, which makes recall@k somewhat optimistic, particularly for short answers.

Groundedness metrics are defined in Section **??**.

4 Task 1: Classical Lexical Retrieve-and-Extract

4.1 BM25 retrieval

BM25 [**?**] scores a passage d for query q by summing, over the query terms, an inverse document frequency weight multiplied by a saturating term-frequency component:

$$\text{BM25}(q, d) = \sum_{t \in q} \text{IDF}(t) \, \frac{f(t, d)\,(k_1 + 1)}{f(t, d) + k_1 \left(1 - b + b \frac{|d|}{\text{avgdl}}\right)}. \tag{2}$$

Rare terms therefore count for more, repeated terms give diminishing returns, and long passages are penalised. We used `rank_bm25.BM25Okapi` with $k_1 = 1.5$ and $b = 0.75$, as specified. Passages and queries are tokenized with `text.lower().split()`.

We also tested a stricter tokenizer that split on punctuation and removed stopwords, on the hypothesis that attached punctuation was preventing matches. It produced worse results on the 300-question rerun, so it was discarded. All results in this report use the original whitespace tokenizer.

4.2 Rule-based answer extraction

Answers are taken only from the top-ranked passage, in two steps:

1. **Sentence selection.** spaCy's `en_core_web_sm` model splits the passage into sentences. The sentence sharing the most normalised word types with the question is selected, and the earliest such sentence wins ties.
2. **Span selection.** The answer is the longest span, by character count, among the sentence's noun chunks and its `CARDINAL` or `DATE` entities. If none exists, the whole sentence is returned.

5 Task 2: Retrieval-Augmented Generation

5.1 The paper: Lewis et al. (2020)

RAG [**?**] addresses a limitation of large language models: their factual knowledge is stored implicitly in their parameters, which makes it hard to inspect, trace to a source, or update. RAG pairs a *parametric* sequence-to-sequence generator with a *non-parametric* retrievable document index, and trains them as one probabilistic model.

Retriever. RAG uses Dense Passage Retrieval (DPR) [**?**], a bi-encoder with separate BERT-based question and passage encoders. A passage's retrieval score is the inner product of the two embeddings:

$$p_\eta(z \mid x) \propto \exp\left(\mathbf{d}(z)^\top \mathbf{q}(x)\right). \tag{3}$$

Because both sides become vectors, a passage can rank highly while sharing few exact words with the question. The paper's knowledge source is a December 2018 Wikipedia dump split into disjoint 100-word chunks, about 21 million documents, indexed with FAISS [**?**] using a Hierarchical Navigable Small World approximation.

Generator. The generator is BART, a pretrained sequence-to-sequence model. The question and retrieved passage are concatenated as its input.

Marginalisation. RAG treats the retrieved document as a latent variable and marginalises over the top k documents:

$$p(y \mid x) \approx \sum_{z \in \text{top-}k} p_\eta(z \mid x)\, p_\theta(y \mid x, z). \tag{4}$$

The paper's two variants differ in where this happens. **RAG-Sequence** uses one document for a whole answer and marginalises over complete hypotheses, while **RAG-Token** can draw on a different document for each generated token. Training minimises the negative marginal log-likelihood by gradient descent, with no explicit document labels, and updates the query encoder and generator; the document encoder and index stay fixed.

Reported results. On Natural Questions, RAG reached 44.1% (Token) and 44.5% (Sequence) EM, above DPR's extractive reader at 41.5% and closed-book T5 at 34.5% [**?**]. The paper also notes that generation can draw on documents that contain clues without stating the answer verbatim, which purely extractive approaches cannot use. Knowledge can also be updated by swapping the index, without retraining.

5.2 Our implementation

Dense retrieval. Passages are embedded with `sentence-transformers/all-MiniLM-L6-v2` [**?**], in batches of 64, and ℓ_2-normalised. They are stored in a FAISS `IndexFlatIP` index, so inner product equals cosine similarity and search is exact. Questions are embedded the same way at query time.

Generation. We used `Qwen/Qwen2.5-1.5B-Instruct` [**?**], an open instruction-tuned model, in 16-bit precision on GPU. The top five passages are each truncated to 200 tokens and inserted into a prompt formatted with the model's chat template:

```
Answer the question based strictly on the provided context. Reply with only
the answer as a short phrase, not a full sentence. If it is a yes/no question,
reply with only yes or no.

Context:
Passage 1: ...
...
Passage 5: ...

Question: {question}
```

The short-answer instruction exists because EM rewards only the exact answer string, and a correct full sentence scores zero. Generation uses at most 64 new tokens with greedy decoding (`do_sample=False`), which is the deterministic equivalent of the specified temperature of 0. Only the newly generated tokens are decoded and scored.

5.3 Differences from the paper

Table **??** summarises how our implementation differs from RAG. The assignment prescribes the retriever and generator family, and these choices also fit the available compute. Conceptually, our system is a *retrieve-then-generate pipeline*, sometimes called "RAG-style", not RAG's jointly trained latent-variable model.

Aspect	Lewis et al. [**?**]	This project
Retriever	DPR bi-encoder	MiniLM-L6-v2, 384-dimensional embeddings
Index	≈21M 100-word chunks; approximate search	45,029 passages of about 100–200 words; exact search
Generator	BART	Qwen2.5-1.5B-Instruct
Training	End-to-end on Natural Questions	None; pretrained models only
Document use	Marginalised over the top k	Top 5 concatenated in one prompt
Decoding	Beam search	Greedy
Evaluation	Official test split	300-question validation sample

Table 2: Key differences between the original RAG system and our reproduction.

6 Task 3: Hybrid Retrieval and Groundedness Verification

6.1 The paper: Cormack et al. (2009)

Rank fusion combines the ranked lists of several retrievers. Score-based methods must reconcile scores on different scales, and supervised learning-to-rank methods need training data. Reciprocal Rank Fusion [**?**] needs neither, because it uses only ranks:

$$\mathrm{RRF}(d) = \sum_{r \in R} \frac{1}{k + r(d)}, \tag{5}$$

where $r(d)$ is the 1-indexed rank of document d in ranking r. The reciprocal form gives top ranks the most influence, while lower ranks keep some effect, unlike under an exponential decay. The constant k limits the influence of a very high rank from an outlier system. The authors fixed $k = 60$ in a pilot study, finding it near-optimal but not critical: mean average precision (MAP) on TREC topics 351–400 varied only from 0.2134 to 0.2147 for k between 20 and 90.

In that pilot, fusing 30 system configurations gave a MAP of 0.2145 at $k = 60$, against 0.2016 for the best individual system, 0.2074 for Condorcet Fuse, and 0.2039 for CombMNZ. Across its pilot and TREC experiments, the paper reports RRF beating Condorcet, CombMNZ, and the best individual system by 4–5% on average. The one exception was TREC 9, where the best run (0.3519 against RRF's 0.2830) used a human in the loop, and RRF still beat the next-best run, which was automated. It also performed strongly as an unsupervised meta-learner on LETOR 3.

6.2 Hybrid retrieval implementation

For each question, BM25 and dense retrieval each rank the *entire* index, so every passage has a rank in both lists. RRF scores are computed with $k = 60$ and equal weights, as specified, and passages are sorted by total score. The top 10 are used to compute recall@k, and the top 5 are passed to the same generator and prompt as Task 2. Both retrievers reuse the Task 1 and Task 2 objects, so the indexes are identical.

The motivation is that the two retrievers should complement each other: BM25 handles rare entity names and numbers, while dense retrieval handles paraphrase.

6.3 Groundedness check

The check reuses Task 1's lexical-overlap idea. Generated answer P and retrieved passages C are normalised, then compared as sets of unigram tokens:

$$\mathrm{Overlap}(P, C) = \frac{|\mathrm{Tokens}(P) \cap \mathrm{Tokens}(C)|}{|\mathrm{Tokens}(P)|}. \tag{6}$$

An empty answer has an overlap of 0. Answers with overlap of at least 0.5 count as grounded, and the rest are flagged as likely ungrounded. We report three measures, with EM defining correctness:

- the fraction of correct answers with high overlap;
- the fraction of incorrect answers with low overlap; and
- groundedness-check accuracy, the share of all answers that are either correct and grounded, or incorrect and flagged.

6.4 Deviations from the specification

- The check is applied to answers generated from the *hybrid* top-5 passages, not Task 2's dense top-5. This evaluates the complete Task 3 system. Task 2's answers were not separately checked, so the statistics describe hybrid output only.
- Flagged answers are reported but not withheld, so the check does not alter Task 3's EM or F1. Section **??** estimates the effect of abstaining.
- Span extraction also considers `DATE` entities, because many Natural Questions answers are dates and the specification's "number span" alone would miss them.

7 Experimental Setup

All 300-question results come from one Slurm batch job on the CSE GPU cluster, with the `gpupart_q4000` partition and one NVIDIA Quadro P4000 (8 GB). The software stack was Python 3.14 and PyTorch 2.14.1. The locked CUDA 13 PyTorch build has no kernels for this GPU, so the job used the CUDA 12.6 build of the same version (`2.14.1+cu126`). This does not change the method. Timestamps on the saved results put Tasks 2 and 3 at about ten minutes each, with embedding the index taking 1 minute 39 seconds of Task 2.

Each run saves its metrics and exact command under `results/`, one folder per task. The results below come from:

```
uv run task --dataset nq_open --sample_size 300 --seed 42
```

8 Results

8.1 Answering and retrieval

Table **??** reports every required metric, and Table **??** gives the same results as question counts. Dense RAG scored highest on every metric.

System	EM	F1	R@1	R@3	R@5	R@10
Task 1: BM25 + extraction	0.33	3.94	25.33	40.33	48.00	56.67
Task 2: Dense + generation	**27.67**	**36.80**	**33.00**	**51.67**	**58.67**	**68.67**
Task 3: Hybrid RRF + generation	25.67	34.15	31.67	47.67	56.00	62.67

Table 3: Results on Natural Questions (Open), $n = 300$, seed 42, in percent. R@k is recall@k.

System	EM	R@1	R@3	R@5	R@10
Task 1	1	76	121	144	170
Task 2	83	99	155	176	206
Task 3	77	95	143	168	188

Table 4: The results in Table **??** as counts out of 300 questions.

Uncertainty. With 300 questions, approximate 95% margins are ±5.1 points for Task 2's EM and ±4.9 for Task 3's, and ±5.3 and ±5.5 for their recall@10. The 2-point EM gap between Tasks 2 and 3 (6 questions) lies well within this noise. The 6-point recall@10 gap (18 questions) is

stronger evidence, though these independent margins do not settle it. All systems share the same questions, so a paired test would be the proper comparison, but it would require per-question outputs, which this run did not save.

8.2 Groundedness verification

Table **??** gives the required metrics, and Table **??** the counts they imply.

Metric	Value
Correct answers with high overlap	100.0%
Incorrect answers with low overlap	11.7%
Groundedness-check accuracy	34.3%

Table 5: Groundedness check on Task 3 answers, threshold 0.5.

	Grounded	Flagged	Total
Correct (EM = 1)	77	0	77
Incorrect (EM = 0)	197	26	223
Total	274	26	300

Table 6: Groundedness outcomes as counts, derived from Table **??** and Task 3's EM.

The check's 34.3% accuracy is low partly because of the metric's construction: treating "grounded" as a prediction of "correct" means that the 197 wrong answers supported by passage vocabulary all count as errors. A check that never flags anything would still score 25.7%. The check's useful property is its precision: all 26 flagged answers were wrong. Abstaining on them would raise the correct share of answered questions from 25.7% to 28.1% (77 of 274) while still answering 91.3% of questions. This is a calculation from the counts and was not a separate run.

8.3 Comparison with the RAG paper

Table **??** compares our result with those of Lewis et al. on Natural Questions. Our Task 2 EM is 16.8 points below RAG-Sequence.

System	NQ EM (%)
RAG-Sequence [**?**]	44.5
RAG-Token [**?**]	44.1
DPR [**?**, **?**]	41.5
T5-11B closed book [**?**]	34.5
Our Task 2 reproduction	**27.7**

Table 7: Published Natural Questions results, as reported in Table 1 of the RAG paper, against our reproduction. The evaluation conditions differ, so this is not a like-for-like comparison.

The likely causes, none of which were isolated experimentally, are:

- **No task-specific training.** This is probably the largest factor. RAG's generator and query encoder are trained end-to-end on Natural Questions, learning its short-answer format. Our models are general-purpose, with a prompt in place of training.

- **Retriever.** DPR's encoders are trained for question-to-passage matching, whereas MiniLM is a compact general-purpose sentence encoder. Passages of up to 200 words may also exceed what MiniLM effectively encodes, so later text may be lost.
- **Generator scale and decoding.** Our 1.5B model reads truncated, concatenated passages and answers with greedy decoding, instead of marginalising over documents with beam search.
- **Index differences.** These work in both directions. Our index is about 470 times smaller and was built around the sampled questions, which should make retrieval easier. However, it is limited to pages that Wikipedia's search returns for those questions, and 13.7% of questions have no gold answer anywhere in the index, which caps recall at 86.3%.
- **Sample.** We used 300 validation questions rather than the full test split, with a margin of about ±5 points.

9 Qualitative Analysis

10 Discussion

Task 1: retrieval was far better than extraction. BM25 found the answer in the top five passages for 48.0% of questions, yet produced just one exact match. The extractor reads only the top passage, whose recall@1 is 25.3%, so most questions fail immediately. Even with the right passage, the longest noun phrase is a poor proxy for the answer: the rule ignores question type, and tends to return long spans where gold answers are short. This is what the specification's fixed heuristic would be expected to do, and is the clearest case for a learned reader.

Coverage bounds every system. With answer coverage at 86.3%, recall cannot exceed that figure. Dense retrieval's recall@10 of 68.7% finds 206 of the 259 answerable questions (79.5%), while BM25's finds 170 (65.6%). Index construction therefore limits results, but most misses within reach still come from ranking.

Task 2: generation closed most of the gap. Moving from extraction to generation raised EM from 0.3% to 27.7%. Retrieval improved too, but that accounts for only a small part of the change: recall@5 rose by 10.7 points. The generator also sees five passages rather than one, and interprets the question. Prompt wording mattered for scoring, since EM penalises any text beyond the answer.

Task 3: fusion did not improve on dense retrieval. This differs from the RRF paper's finding, and the data suggest why. Hybrid recall sits between BM25's and dense retrieval's at every k, and the fused system's largest deficit is at $k = 10$, at 6.0 points. With equal weights and two lists, a passage that ranks moderately in both can overtake one ranked highly by the stronger retriever alone. Since BM25 is clearly weaker here at every k, it may displace good dense results more often than it adds new answers.

Two differences from the paper may explain this. RRF was evaluated on many runs, such as 30 system configurations, where no single ranking dominates; fusing just two rankings of unequal quality is a much weaker setting. BM25 here is also deliberately simple, with whitespace tokenization. However, the stricter tokenizer we tried did not help, so this weakness has no easy fix. Our reader-level gap was small: the EM-to-recall@5 ratio was 47% for Task 2 and 46% for Task 3, which suggests that the hybrid deficit stems mainly from retrieval.

Groundedness: overlap measures vocabulary, not truth. The check confirmed that every correct answer was supported by passage vocabulary, and that every flagged answer was

wrong, but it caught only about one wrong answer in nine. Three design limits explain this. The generator is told to answer from the context, and so naturally copies passage words. The token set of five 200-word passages is large, so even wrong answers overlap by chance. Unigram overlap also ignores whether an answer's words appear together or answer the question. The check therefore works best as a high-precision warning for unsupported output, and not as a general verifier of correctness.

Limitations.

- The index is specific to our question sample and differs from a general Wikipedia index. It also lacks any gold answer string for 13.7% of questions.
- Recall@k checks for answer strings, not actual evidence.
- EM marks correct paraphrases as wrong, so it understates true accuracy.
- Differences of a few points are within sampling noise at $n = 300$.
- The results come from a single seed and a single frozen index.

Future work. Promising directions include: weighted RRF favouring the dense retriever; a cross-encoder reranker over the fused candidates; a stronger dense encoder; retrieving more candidates before generation; a few-shot prompt drawn from the Natural Questions training split, which the assignment allows; an entailment-based groundedness check, along with a threshold sweep; and logging per-question outputs to allow paired significance testing.

11 Conclusion

We built and compared three open-domain QA systems on a shared Wikipedia-derived index. Dense retrieval with generation was the most effective, achieving 27.7% EM and 68.7% recall@10, far above the rule-based extractive baseline's 0.3% EM. Our EM was below the 44.5% reported for end-to-end trained RAG, a gap consistent with the absence of task-specific training and the smaller models used.

Hybrid RRF did not deliver the expected improvement, with 25.7% EM and 62.7% recall@10, which suggests that equal-weight fusion is less useful when one retriever is clearly weaker. The lexical groundedness check flagged only wrong answers, but only 11.7% of them. It can serve as a precise warning signal, but cannot on its own establish that an answer is correct. Overall, for this dataset and index, the largest gains came from replacing rule-based extraction with generation, not from combining retrievers.

References

[1] D. Chen, A. Fisch, J. Weston, and A. Bordes. Reading Wikipedia to answer open-domain questions. In *Proc. ACL*, 2017. arXiv:1704.00051.

[2] P. Lewis, E. Perez, A. Piktus, et al. Retrieval-augmented generation for knowledge-intensive NLP tasks. In *Proc. NeurIPS*, 2020. arXiv:2005.11401.

[3] G. V. Cormack, C. L. A. Clarke, and S. Büttcher. Reciprocal rank fusion outperforms Condorcet and individual rank learning methods. In *Proc. SIGIR*, 2009.

[4] V. Karpukhin, B. Oğuz, S. Min, et al. Dense passage retrieval for open-domain question answering. In *Proc. EMNLP*, 2020.

[5] S. Robertson and H. Zaragoza. The probabilistic relevance framework: BM25 and beyond. *Foundations and Trends in Information Retrieval*, 3(4), 2009.

[6] T. Kwiatkowski, J. Palomaki, O. Redfield, et al. Natural Questions: A benchmark for question answering research. *TACL*, 7, 2019.

[7] K. Lee, M.-W. Chang, and K. Toutanova. Latent retrieval for weakly supervised open domain question answering. In *Proc. ACL*, 2019.

[8] P. Rajpurkar, J. Zhang, K. Lopyrev, and P. Liang. SQuAD: 100,000+ questions for machine comprehension of text. In *Proc. EMNLP*, 2016.

[9] N. Reimers and I. Gurevych. Sentence-BERT: Sentence embeddings using Siamese BERT-networks. In *Proc. EMNLP*, 2019.

[10] J. Johnson, M. Douze, and H. Jégou. Billion-scale similarity search with GPUs. *IEEE Transactions on Big Data*, 2019.

[11] Qwen Team. Qwen2.5 technical report. arXiv:2412.15115, 2024.